

Engenharia de Software II

Introdução ao pytest (Parte 2)

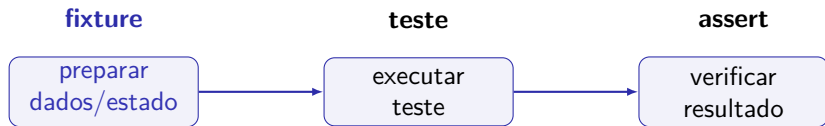
Vinicius H. S. Durelli

✉ `vinicius.durelli@ufscar.br`



Fixtures em pytest

➡ Agora que já sabemos escrever e executar funções de teste, vamos estudar um mecanismo fundamental para estruturar testes em sistemas um pouco maiores: **fixtures**.



Fixtures ajudam a deixar os testes mais organizados, reutilizáveis e fáceis de manter.

Por que precisamos de fixtures?

➡ Em exemplos pequenos, cada teste pode preparar seus próprios dados. Mas, em sistemas reais, essa preparação tende a se repetir.

Sem fixtures

- ➡ preparação repetida;
- ➡ testes mais longos;
- ➡ maior chance de inconsistência;
- ➡ manutenção mais difícil.

Com fixtures

- ➡ preparação centralizada;
- ➡ testes mais curtos;
- ➡ dados reutilizáveis;
- ➡ estado conhecido antes do teste.

Uma fixture pode criar dados, configurar objetos, preparar arquivos, abrir conexões, inicializar recursos ou colocar o sistema em um estado conhecido antes da execução do teste.

O que vamos estudar nesta parte?

➡ Fixtures ajudam a estruturar testes automatizados com pytest, especialmente quando há preparação de dados, estado compartilhado ou recursos externos.

Conceitos principais

- ➡ criação de fixtures;
- ➡ uso de fixtures em testes;
- ➡ setup e teardown;
- ➡ escopo de fixtures.

Aspectos práticos

- ➡ reutilização de dados;
- ➡ múltiplas fixtures;
- ➡ execução antes/depois dos testes;
- ➡ rastreamento da execução.

Objetivo: escrever testes mais limpos, menos repetitivos e mais fáceis de evoluir.

Uma primeira fixture

```
import pytest

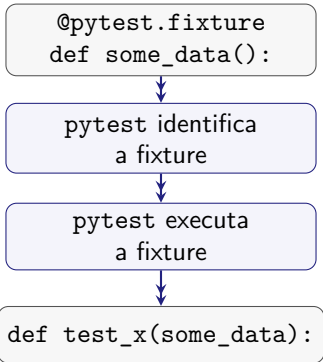
@pytest.fixture
def carrinho_vazio():
    return Carrinho()

def test_carrinho_comeca_vazio(carrinho_vazio):
    assert len(carrinho_vazio) == 0
```

A fixture `carrinho_vazio` prepara o objeto que será usado pelo teste. O `pytest` executa a fixture automaticamente antes da função de teste.

O que é uma fixture?

➡ Uma **fixture** é uma função auxiliar usada para preparar algo que um ou mais testes precisam para executar.



Quando o nome da fixture aparece como parâmetro de uma função de teste, o `pytest` procura uma fixture com esse nome, executa essa função antes do teste e entrega ao teste o valor retornado por ela.

O que uma fixture pode preparar?

O termo **fixture** pode aparecer com sentidos um pouco diferentes, mas aqui vamos usá-lo principalmente para falar de funções decoradas com `@pytest.fixture`.

No contexto do pytest

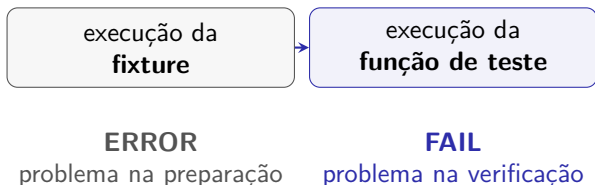
Uma fixture é uma função que prepara ou recupera dados, objetos ou recursos que serão usados por uma ou mais funções de teste.

- ▶ Ela pode criar um objeto que será usado pelo teste;
- ▶ Ela pode colocar o sistema em um estado conhecido antes da execução;
- ▶ Ela pode carregar dados, abrir arquivos, preparar diretórios ou configurar algum recurso externo;
- ▶ Ela também pode retornar dados diretamente para a função de teste.

Resumo: fixtures separam a *preparação* do teste da *verificação* feita pelo teste.

Falha no teste ou erro na fixture?

 O pytest diferencia problemas que acontecem **dentro da função de teste** de problemas que acontecem **durante a execução de uma fixture**.



- ➡ Se uma exceção, um `assert` ou uma chamada a `pytest.fail()` acontece dentro da fixture, o resultado é reportado como **ERROR**.
- ➡ Se o problema acontece dentro da função de teste, o resultado é reportado como **FAIL**.

Fixtures para setup e teardown

- ▶▶ Fixtures são especialmente úteis quando os testes precisam preparar algum estado antes da execução e limpar recursos depois.

Em uma aplicação real, os testes raramente verificam apenas funções isoladas. Muitas vezes, eles precisam interagir com objetos, arquivos, bancos de dados ou outros recursos que precisam ser preparados antes do teste.

- ▶▶ Para testar uma aplicação com banco de dados, precisamos começar cada teste em um estado conhecido.
- ▶▶ Se vários testes compartilham a mesma preparação, repetir esse código em cada teste torna a suíte mais frágil.
- ▶▶ Fixtures permitem concentrar essa preparação em funções auxiliares reutilizáveis.

Setup → preparar o estado antes do teste.

Teardown → limpar depois do teste.

Observando o comportamento pela linha de comando

➡ Antes de escrever os testes, vamos observar o comportamento esperado da aplicação pela linha de comando.

Execução no terminal

```
$ cards count  
0  
$ cards add first item  
$ cards add second item  
$ cards count  
2
```

A funcionalidade `count` informa quantos `cards` existem no momento. Depois de adicionar dois itens, esperamos que o comando `cards count` retorne 2.

Primeiro teste para count()

➡ Vamos começar testando o caso mais simples: uma base recém-criada deve conter zero cards.

```
from pathlib import Path
from tempfile import TemporaryDirectory
import cards

def test_empty():
    with TemporaryDirectory() as db_dir:
        db_path = Path(db_dir)
        db = cards.CardsDB(db_path)

        count = db.count()
        db.close()

        assert count == 0
```

O que esse teste faz?

➡ Para chamar `count()`, precisamos primeiro criar um objeto que represente a base de dados da aplicação.

`TemporaryDirectory()` cria um diretório temporário



`Path(db_dir)` transforma o caminho em um objeto `Path`



`cards.CardsDB(db_path)` cria o objeto de banco



`db.count()` retorna a quantidade de cards



`db.close()` fecha a base de dados

O teste verifica uma propriedade simples: se a base acabou de ser criada, então `db.count()` deve retornar 0.

Mas há um problema de estrutura

- ➡ O teste funciona, mas mistura duas responsabilidades: preparar a base de dados e verificar o comportamento de `count()`.

Preparação dentro do teste

- ➡ criar um diretório temporário;
- ➡ converter o caminho para `Path`;
- ➡ criar o objeto `CardsDB`;
- ➡ fechar a base com `db.close()`.

Observe que `db.close()` aparece antes do `assert`. À primeira vista, faria sentido fechar a base no final da função. Porém, se o `assert` falhar, as instruções posteriores não serão executadas. Por isso, neste teste, a base precisa ser fechada antes da verificação.

Resolvendo com uma fixture

➡ Agora vamos mover a preparação da base de dados para uma fixture:

```
from pathlib import Path
from tempfile import TemporaryDirectory
import pytest
import cards

@pytest.fixture
def cards_db():
    with TemporaryDirectory() as db_dir:
        db_path = Path(db_dir)
        db = cards.CardsDB(db_path)
        yield db
        db.close()

def test_empty(cards_db):
    assert cards_db.count() == 0
```

O que mudou?

➡ A **função de teste ficou menor e mais fácil de ler**, pois toda a inicialização da base foi movida para a fixture `cards_db`.

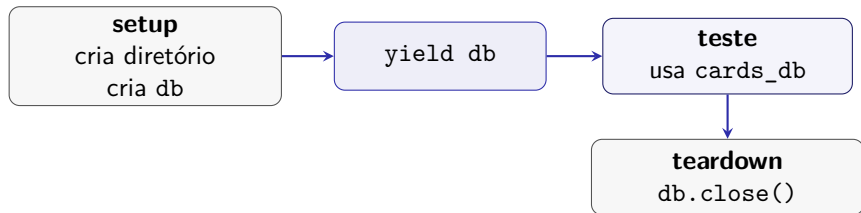
Leitura do código

- ➡ A fixture `cards_db()` prepara a base de dados para o teste.
- ➡ A instrução `yield db` entrega o objeto `db` à função de teste.
- ➡ O teste `test_empty(cards_db)` recebe esse objeto pelo nome da fixture.
- ➡ Assim, a função de teste passa a focar apenas na verificação: `cards_db.count() == 0`.

Ideia central: a fixture cuida da preparação; o teste cuida da verificação.

yield: *setup* e *teardown*

➡ Quando uma fixture usa `yield`, o código antes dele é executado antes do teste, e o código depois dele é executado quando o teste termina.



➡ O código **antes** do `yield` corresponde ao **setup**. O código **depois** do `yield` corresponde ao **teardown**. Isso é útil porque a limpeza (i.e., o `db.close()`) fica associada à fixture, e não espalhada pelos testes.

Importante

O pytest olha para o **nome dos argumentos** da função de teste e procura uma fixture com o **mesmo nome**.

```
def test_empty(cards_db):
```

Nós **não chamamos fixtures diretamente**. Quem faz isso é o próprio pytest.

Reutilizando a mesma fixture...

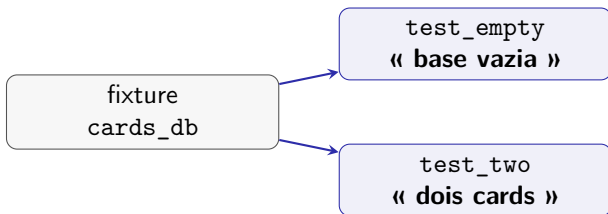
➡ **A mesma fixture pode ser usada por vários testes.** Cada teste recebe uma base preparada pela fixture `cards_db`.

```
def test_two(cards_db):  
    cards_db.add_card(cards.Card("first"))  
    cards_db.add_card(cards.Card("second"))  
  
    assert cards_db.count() == 2
```

➡ Neste teste, começamos com uma base vazia, adicionamos dois cards e verificamos se `count()` retorna 2.

O teste fica focado no comportamento

- ▶ Como a preparação está encapsulada na fixture, o teste pode se concentrar apenas no comportamento que queremos verificar.



- ▶ `test_empty()` verifica o caso em que a base começa vazia.
- ▶ `test_two()` verifica o caso em que dois cards foram adicionados.
- ▶ Ambos reutilizam a mesma preparação: uma base configurada e pronta para uso.

Rastreando a execução da fixture

➡ Quando vários testes usam a mesma fixture, pode ser útil observar exatamente em que ordem tudo é executado.

Opção útil do pytest

A flag `--setup-show` mostra a ordem de execução dos testes e fixtures, incluindo as fases de **setup** e **teardown**.

Execução no terminal

```
$ cd /path/to/code  
$ pytest --setup-show test_count.py
```

Saída produzida por `--setup-show`

Execução no terminal

```
$ pytest --setup-show test_count.py
===== test session starts =====
collected 2 items
test_count.py
  SETUP      F cards_db
  path/test_count.py::test_empty (fixtures used:  cards_db).
  TEARDOWN   F cards_db
  SETUP      F cards_db
  path/test_count.py::test_two (fixtures used:  cards_db).
  TEARDOWN   F cards_db

===== 2 passed in 0.02s =====
```

Escopo de fixtures

➡ O **escopo** de uma fixture define com que frequência seu setup e seu teardown são executados.

Escopo padrão

Por padrão, fixtures têm escopo de **função**. Isso significa que o setup roda antes de cada teste que usa a fixture, e o teardown roda depois de cada um desses testes.

test_empty ⇒ setup → teste → teardown

test_two ⇒ setup → teste → teardown

Quando mudar o escopo?

➡ Às vezes, preparar a fixture pode ser caro. Nesses casos, talvez não seja desejável repetir o setup para cada teste.

- ➡ Conectar a um banco de dados;
- ➡ Gerar uma grande massa de dados;
- ➡ Baixar dados de um servidor;
- ➡ Inicializar um recurso lento.

Ideia

Se vários testes podem compartilhar a mesma preparação, podemos alterar o escopo da fixture para executar o setup uma única vez para um grupo maior de testes.

Alterando o escopo da fixture

➡ A mudança é pequena: adicionamos `scope="module"` ao *decorator* da fixture.

```
@pytest.fixture(scope="module")
def cards_db():
    with TemporaryDirectory() as db_dir:
        db_path = Path(db_dir)
        db = cards.CardsDB(db_path)
        yield db
        db.close()
```

➡ Com escopo de módulo, a fixture pode ser criada uma vez e reutilizada pelos testes do mesmo arquivo.

Executando com escopo de módulo

➡ Agora a fixture `cards_db` é configurada uma única vez para todos os testes deste módulo.

Execução no terminal

```
$ pytest --setup-show test_mod_scope.py
===== test session starts =====
collected 2 items
test_mod_scope.py
  SETUP      M cards_db
    path/test_mod_scope.py::test_empty (fixtures used: cards_db).
    path/test_mod_scope.py::test_two (fixtures used: cards_db).
  TEARDOWN M cards_db

===== 2 passed in 0.03s =====
```

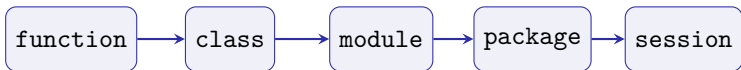
➡ Com `scope="module"`, **os dois testes compartilham a mesma instância da fixture**. O setup acontece antes do primeiro teste do módulo, e o teardown acontece somente depois do último teste que usa essa fixture.

Escopos de fixtures no pytest

- ➡ O parâmetro `scope` do decorador `@pytest.fixture` define **com que frequência** a fixture será criada e destruída.

Escopos disponíveis

O pytest oferece os seguintes valores para `scope`: "function", "class", "module", "package" e "session".



Da esquerda para a direita, aumenta o compartilhamento da fixture. O escopo padrão é `function`.

Escopos mais comuns

➡ Os três primeiros escopos costumam ser os mais frequentes no dia a dia.

`scope="function"`

- ➡ Executa uma vez por função de teste;
- ➡ O setup roda antes de cada teste que usa a fixture;
- ➡ O teardown roda depois de cada teste;
- ➡ É o escopo padrão.

`scope="class"`

Executa uma vez por classe de teste, independentemente da quantidade de métodos de teste dentro dela.

`scope="module"`

Executa uma vez por módulo (ou seja, por arquivo de teste), independentemente de quantas funções ou métodos usam a fixture.

Escopos mais amplos

➡ Quando queremos compartilhar uma fixture por um conjunto ainda maior de testes, podemos usar escopos mais amplos.

`scope="package"`

Executa uma vez por pacote (ou diretório de testes), independentemente de quantas funções, métodos ou fixtures dentro desse pacote a utilizam.

`scope="session"`

Executa uma vez por sessão de testes inteira. Todas as funções e métodos que usam essa fixture compartilham o mesmo setup e o mesmo teardown.

Regra prática

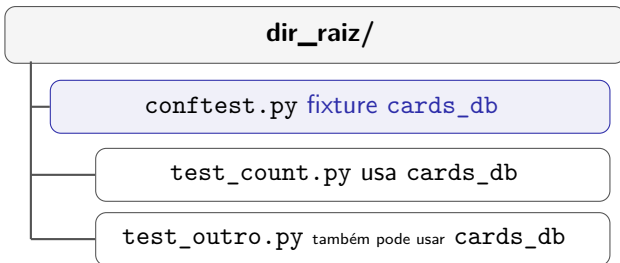
Quanto maior o escopo, menor o número de execuções do setup/teardown. Em compensação, maior o compartilhamento de estado entre os testes.

Compartilhando fixtures com conftest.py

➡ Até agora, colocamos a fixture no mesmo arquivo dos testes. Mas, quando vários arquivos de teste precisam da mesma fixture, podemos movê-la para um arquivo `conftest.py`.

`conftest.py`

O arquivo `conftest.py` é descoberto automaticamente pelo `pytest`. Fixtures definidas nele ficam disponíveis para os testes no mesmo diretório e, dependendo da estrutura do projeto, também em diretórios abaixo dele.



A fixture em conftest.py

➡ Vamos mover a fixture `cards_db` para um arquivo `conftest.py`.

```
from pathlib import Path
from tempfile import TemporaryDirectory
import cards
import pytest

@pytest.fixture(scope="session")
def cards_db():
    """CardsDB object connected to a temporary database"""
    with TemporaryDirectory() as db_dir:
        db_path = Path(db_dir)
        db = cards.CardsDB(db_path)
        yield db
        db.close()
```

➡ O `pytest` trata `conftest.py` como um tipo de plugin local: ele pode conter fixtures e outras configurações usadas pelos testes.

O arquivo de teste fica mais simples

➡ Depois que a fixture foi movida para `conftest.py`, o arquivo de teste não precisa mais defini-la.

```
import cards

def test_empty(cards_db):
    assert cards_db.count() == 0

def test_two(cards_db):
    cards_db.add_card(cards.Card("first"))
    cards_db.add_card(cards.Card("second"))

    assert cards_db.count() == 2
```

➡ O teste continua recebendo `cards_db` pelo nome, mas agora a fixture vem de `conftest.py`.

Encontrando onde fixtures são definidas

➡ Fixtures podem estar no próprio arquivo de teste ou em arquivos `conftest.py` no mesmo diretório ou em diretórios pais.

Opção útil do pytest

Use `pytest --fixtures -v` para listar as fixtures disponíveis, incluindo o arquivo e a linha onde cada fixture foi definida.

Execução no terminal

```
$ cd /path/to/code/
$ pytest --fixtures -v
...
----- fixtures defined from conftest -----
cards_db [session scope] - conftest.py:7
    CardsDB object connected to a temporary database
...
```

Resumo: quando não lembrar onde uma fixture está definida, peça ao próprio pytest para mostrar.

Vendo quais fixtures um teste usa...

➡ Se quisermos saber quais fixtures são usadas por um teste específico, podemos usar a opção `--fixtures-per-test`.

Execução no terminal

```
$ pytest --fixtures-per-test test_count.py::test_empty
===== test session starts =====
collected 1 item

----- fixtures used by test_empty -----
----- (test_count.py:4) -----
cards_db - conftest.py:7
    CardsDB object connected to a temporary database

===== no tests ran in 0.00s =====
```

Leitura da saída

Neste exemplo, pedimos informações sobre o teste `test_count.py::test_empty`. O `pytest` mostra que esse teste usa a fixture `cards_db`, definida em `conftest.py`.

- ➔ Quando usamos uma fixture com escopo de `module` ou `session`, vários testes podem compartilhar a mesma instância do recurso.
- ➔ Isso pode ser um problema quando os testes assumem que o banco de dados começa **vazio**, mas na prática estão reutilizando a mesma instância já modificada por testes anteriores.

```
def test_three(cards_db):  
    cards_db.add_card(cards.Card("first"))  
    cards_db.add_card(cards.Card("second"))  
    cards_db.add_card(cards.Card("third"))  
  
    assert cards_db.count() == 3
```

Execução no terminal

```
$ pytest -v test_three.py  
===== test session starts =====  
collected 1 item  
test_three.py::test_three PASSED [100%]  
  
===== 1 passed in 0.01s =====
```

O problema aparece ao executar a suíte

Execução no terminal

```
$ pytest -v --tb=line test_count.py test_three.py
===== test session starts =====
collected 3 items

test_count.py::test_empty PASSED [ 33%]
test_count.py::test_two PASSED [ 66%]
test_three.py::test_three FAILED [100%]

===== FAILURES =====
/path/to/code/test_three.py:8: assert 5 == 3
===== short test summary info
=====
FAILED test_three.py::test_three - assert 5 == 3
===== 1 failed, 2 passed in 0.01s =====
```

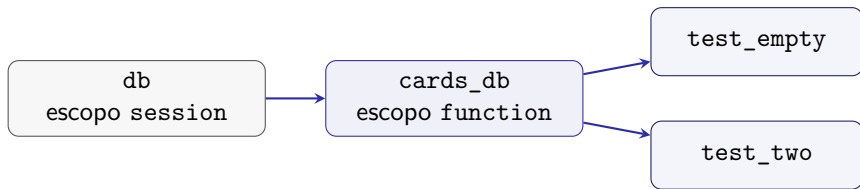
➡ O teste `test_three()` espera encontrar apenas os três cards que ele adicionou. Porém, como a fixture está compartilhando a mesma base, o teste recebe também os dois cards adicionados por `test_two()`.

Combinando múltiplos níveis de fixture

➡ Queremos manter **uma única base aberta** durante toda a sessão de testes, mas ainda garantir que **cada teste comece com zero elementos**.

Ideia

Podemos resolver isso usando **duas fixtures com escopos diferentes**: uma fixture mais ampla para abrir a base apenas uma vez, e outra fixture mais restrita para limpar o conteúdo antes de cada teste.



📌 **Resultado:** reaproveitamos o recurso caro, mas evitamos estado compartilhado entre os testes.

Definindo as duas fixtures

➡ A solução é renomear a fixture antiga para db e criar uma nova fixture cards_db que depende dela.

```
@pytest.fixture(scope="session")
def db():
    """CardsDB object connected to a temporary database"""
    with TemporaryDirectory() as db_dir:
        db_path = Path(db_dir)
        db_ = cards.CardsDB(db_path)
        yield db_
        db_.close()

@pytest.fixture(scope="function")
def cards_db(db):
    """CardsDB object that is empty"""
    db.delete_all()
    return db
```

Como essa solução funciona?

- ➡ Agora cada fixture tem uma responsabilidade bem definida.

Leitura da solução

- ➡ db tem escopo `session`: a base é aberta uma única vez para toda a sessão de testes.
- ➡ `cards_db` tem escopo `function`: ela é executada antes de cada teste que a utiliza.
- ➡ A chamada `db.delete_all()` limpa a base antes de entregar o objeto ao teste.
- ➡ Assim, os testes compartilham a mesma conexão com a base, mas não compartilham os dados deixados por testes anteriores.

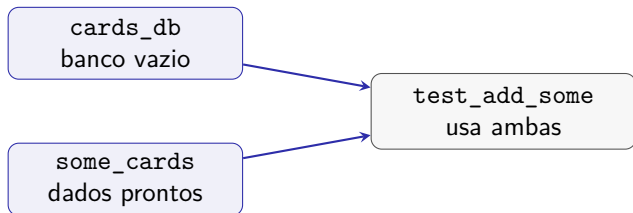
A fixture `db` cuida do recurso de longa duração. A fixture `cards_db` cuida do **estado inicial** esperado por cada teste.

Usando múltiplas fixtures

➡ Até agora, cada teste usava apenas uma fixture. Mas o pytest permite que uma função de teste (ou mesmo outra fixture) dependa de **várias fixtures ao mesmo tempo**.

Ideia principal

Podemos combinar fixtures com responsabilidades diferentes. Por exemplo: uma fixture pode preparar um banco vazio, enquanto outra fornece um conjunto de dados prontos para serem usados no teste.



Resultado: o teste passa a combinar recurso + dados de entrada.

Uma fixture com dados prontos

➡ Podemos definir uma fixture que retorna uma lista de objetos Card para serem reutilizados em vários testes.

```
@pytest.fixture(scope="session")
def some_cards():
    """List of different Card objects"""
    return [
        cards.Card("write book", "Brian", "done"),
        cards.Card("edit book", "Katie", "done"),
        cards.Card("write 2nd edition", "Brian", "todo"),
        cards.Card("edit 2nd edition", "Katie", "todo"),
    ]
```

➡ A fixture `some_cards` encapsula um conjunto fixo de dados. Assim, não precisamos recriar manualmente esses objetos em cada teste.

Um teste usando duas fixtures

➡ Agora o teste pode receber simultaneamente o banco de dados vazio e a lista de cards.

```
def test_add_some(cards_db, some_cards):  
    expected_count = len(some_cards)  
  
    for c in some_cards:  
        cards_db.add_card(c)  
  
    assert cards_db.count() == expected_count
```

Leitura do teste

A fixture `cards_db` fornece um banco vazio. A fixture `some_cards` fornece os dados de entrada. O teste apenas insere os cards e verifica se a contagem final é igual ao número de elementos da lista.

Fixtures também podem usar fixtures

- ➡ Além de testes usarem múltiplas fixtures, uma fixture também pode depender de outras fixtures.

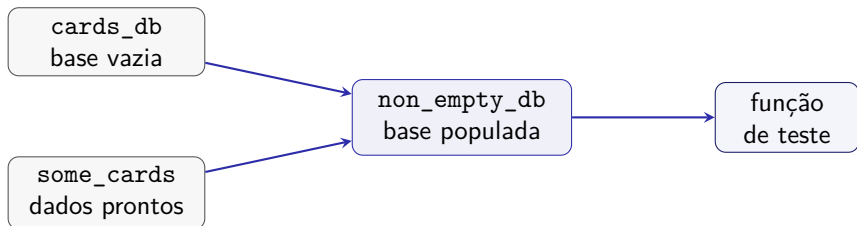
```
@pytest.fixture(scope="function")
def non_empty_db(cards_db, some_cards):
    """CardsDB that's been populated with 'some_cards'"""
    for c in some_cards:
        cards_db.add_card(c)

    return cards_db
```

Leitura

A fixture `non_empty_db` recebe duas fixtures: `cards_db`, que fornece uma base vazia, e `some_cards`, que fornece os dados a serem inseridos.

Composição de fixtures



- `cards_db` prepara uma base vazia.
- `some_cards` fornece uma coleção reutilizável de objetos.
- `non_empty_db` combina as duas e entrega ao teste uma base já populada.

Cuidado com escopo

Uma fixture não pode depender de outra fixture com escopo mais restrito.

Usando a fixture composta

➡ Agora, testes que precisam de uma base já populada podem simplesmente utilizar a fixture `non_empty_db`.

```
@pytest.fixture(scope="function")
def non_empty_db(cards_db, some_cards):
    """CardsDB that's been populated with 'some_cards'"""
    for c in some_cards:
        cards_db.add_card(c)

    return cards_db
```

```
def test_non_empty(non_empty_db):
    assert non_empty_db.count() > 0
```

Exercício ①

Parte 1: criando fixtures simples

- Crie um arquivo de testes chamado `test_fixtures.py`.
- Escreva algumas fixtures de dados usando o decorador `@pytest.fixture()`.
- Cada fixture deve retornar algum dado simples, como uma lista, um dicionário ou uma tupla.
- Para cada fixture, escreva pelo menos uma função de teste que a utilize.

Exercício ②

Parte 2: reutilizando fixtures

- Escolha uma das fixtures criadas anteriormente.
- Escreva dois testes que utilizem essa mesma fixture.
- Execute o comando `pytest --setup-show test_fixtures.py`.
- Observe se a fixture é executada antes de cada teste.

Exercício ③

Parte 3: alterando o escopo da fixture

- Na fixture usada por dois testes, adicione o parâmetro `scope="module"` ao decorador.
- Execute novamente o comando `pytest --setup-show test_fixtures.py`.
- Compare a nova saída com a saída obtida anteriormente.
- O que mudou na quantidade de vezes que a fixture foi executada?

Exercício ④

Parte 4: usando yield

- Na mesma fixture, substitua o `return` por `yield`.
- Adicione um `print()` antes do `yield`.
- Adicione outro `print()` depois do `yield`.
- Execute o comando `pytest -s -v test_fixtures.py`.